

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Trabajo Práctico Integrador

Programación 2

FOOD STORE

Sistema de Gestión de Pedidos de Comida (Consola)

Estudiante:	DALESIO, Gerardo Martin
Grupo	52
Legajo:	101869
Materia:	Programación 2
Carrera:	Tecnicatura Universitaria en Programación
Modalidad:	A Distancia

2026

Índice

Índice.....	i
1. Marco Teórico	1
1.1 Java.....	1
1.2 Persistencia en memoria (sin base de datos relacional).....	1
1.3 Conceptos de POO aplicados	1
2. Decisiones Técnicas y Arquitectura	2
2.1 Organización de paquetes	2
2.2 Flujo de una operación típica.....	2
2.3 Generación de identificadores.....	2
2.4 Resolución de la dependencia circular Categoría–Producto	2
2.5 Creación de Pedidos en tres pasos	3
2.6 Excepciones propias.....	3
2.7 Baja lógica (soft delete).....	3
2.8 Capturas de pantalla del sistema funcionando.....	3
3. Dificultades y Soluciones.....	6
3.1 Generación de identificadores en clases de dominio	6
3.2 Dependencia circular entre Categoría y Producto.....	7
3.3 Consistencia de las colecciones al crear un pedido con detalles.....	7
3.4 Validación de unicidad de campos (nombre de categoría, mail de usuario).....	7
3.5 Programación Genérica para Evitar Duplicación.....	7
4. Bibliografía / Webgrafía	8
5. Enlaces del Proyecto	8

1. Marco Teórico

El presente Trabajo Práctico Integrador se desarrolló utilizando el lenguaje de programación Java (versión 21), bajo el paradigma de Programación Orientada a Objetos (POO). A continuación se describen brevemente las tecnologías y conceptos clave aplicados durante el desarrollo.

1.1 Java

Java es un lenguaje de programación orientado a objetos, fuertemente tipado y multiplataforma, ejecutado sobre la Máquina Virtual de Java (JVM). Se eligió por ser el lenguaje de cátedra y por su robusto soporte nativo para Colecciones, manejo de excepciones e interfaces, todos ellos requisitos centrales de la consigna.

1.2 Persistencia en memoria (sin base de datos relacional)

La consigna del presente trabajo establece explícitamente que el sistema no debe utilizar una base de datos, sino que toda la información se almacena en memoria durante la ejecución mediante el uso de Colecciones (`java.util.ArrayList`). Esta decisión simplifica el alcance del proyecto y permite concentrar el desarrollo en los conceptos de POO, Colecciones, Interfaces y Excepciones, que son los objetivos de aprendizaje definidos para Programación 2.

1.3 Conceptos de POO aplicados

- Encapsulamiento: todos los atributos de las entidades son privados, accesibles mediante getters y setters.
- Herencia: las clases `Categoria`, `Producto`, `Usuario`, `Pedido` y `DetallePedido` heredan de una clase abstracta `Base`, que centraliza los atributos comunes (`id`, `eliminado`, `createdAt`).
- Polimorfismo e Interfaces: la clase `Pedido` implementa la interfaz `Calculable`, que define el método `calcularTotal()`, permitiendo que el cálculo del total se invoque de manera uniforme.
- Colecciones: se utiliza `ArrayList<T>` para almacenar y gestionar en memoria cada tipo de entidad dentro de su respectivo `Service`.
- Excepciones propias: se definieron las clases `EntidadNoEncontradaException` y `StockInvalidoException` para representar errores específicos del dominio del negocio.

2. Decisiones Técnicas y Arquitectura

2.1 Organización de paquetes

El proyecto se organizó siguiendo la arquitectura recomendada por la cátedra, separando responsabilidades en capas bien definidas:

- **main:** contiene la clase `Main.java`, único punto de entrada del programa. Su responsabilidad se limita a instanciar los `Services` y el `MenuPrincipal`, sin contener lógica de negocio.
- **menu:** contiene las clases de interacción con el usuario (`MenuPrincipal`, `CategoriaMenu`, `ProductoMenu`, `UsuarioMenu`, `PedidoMenu`). Se encargan exclusivamente de leer datos por consola (`Scanner`) y mostrar resultados o mensajes de error.
- **services:** contiene la lógica de negocio (`CategoriaServicio`, `ProductoServicio`, `UsuarioServicio`, `PedidoServicio`) y la clase auxiliar `Validaciones`. Aquí se aplican las reglas de validación, generación de identificadores y baja lógica.
- **entities:** contiene el modelo de dominio (`Base`, `Categoria`, `Producto`, `Usuario`, `Pedido`, `DetallePedido`) y la interfaz `Calculable`.
- **enums:** contiene los enumerados `Rol`, `Estado` y `FormaPago`.
- **exceptions:** contiene las excepciones propias del dominio.

2.2 Flujo de una operación típica

Para ilustrar la separación de capas, se describe el flujo de la operación “crear una categoría”: el usuario interactúa con `CategoriaMenu`, que solicita nombre y descripción por consola, sin validar nada por sí mismo. Estos datos se envían a `CategoriaServicio.crear()`, que valida que el nombre no esté vacío, que no exista otra categoría activa con el mismo nombre, genera el identificador correspondiente mediante un contador propio del `Service`, y finalmente construye y persiste el objeto `Categoria` en la colección en memoria. Si alguna validación falla, el `Service` lanza una excepción que es capturada por el `Menu`, quien informa el error al usuario sin interrumpir la ejecución del programa.

2.3 Generación de identificadores

Cada entidad recibe su identificador (`id`) desde el `Service` correspondiente, el cual mantiene un contador propio de instancia (no estático). Se evitó deliberadamente el uso de contadores estáticos dentro de las clases de dominio, dado que esto acopla la lógica de generación de identidad —una decisión de persistencia— a la entidad, que debería ocuparse únicamente de representar el concepto de negocio. Esta decisión surgió a partir de una observación recibida sobre la primera entrega parcial del proyecto.

2.4 Resolución de la dependencia circular Categoría–Producto

`ProductoServicio` necesita una referencia a `CategoriaServicio` para validar que la categoría indicada exista al crear o editar un producto. A su vez, `CategoriaServicio` necesita una referencia a

ProductoServicio para verificar, antes de eliminar una categoría, si existen productos activos asociados a ella. Esta dependencia circular se resolvió instanciando primero CategoriaServicio (sin dependencias), luego ProductoServicio (que lo recibe por constructor), y finalmente conectando la referencia inversa mediante el método categoriaServicio.setProductoServicio(productoServicio) desde la clase Main.

2.5 Creación de Pedidos en tres pasos

La creación de un Pedido con sus respectivos detalles se modeló en tres pasos diferenciados: iniciarPedido() construye el objeto Pedido en memoria pero todavía no lo agrega a la colección definitiva; agregarDetalle() permite cargar uno o más productos con su cantidad, validando stock disponible en cada caso; y confirmarPedido() recién en este punto incorpora el pedido a la colección de pedidos del sistema. Si durante la carga de detalles se produce un error (por ejemplo, stock insuficiente), el pedido se descarta sin haber llegado nunca a formar parte de la colección, evitando así dejar datos inconsistentes en memoria.

2.6 Excepciones propias

Se definieron dos excepciones propias, EntidadNoEncontradaException y StockInvalidoException, ambas extendiendo de RuntimeException (excepciones no verificadas). Esta decisión se tomó para simplificar la propagación de errores entre las capas de Service y Menu, sin obligar a declarar throws en cada método intermedio. No obstante, ambas excepciones se capturan explícitamente mediante bloques try/catch en la capa de Menu, garantizando que el usuario reciba siempre un mensaje de error comprensible en lugar de una interrupción abrupta del programa.

2.7 Baja lógica (soft delete)

Todas las entidades del sistema heredan de la clase abstracta Base, que define el atributo eliminado. Ningún método eliminar() de los distintos Services remueve físicamente un objeto de su colección; en su lugar, se marca el atributo eliminado en true. De esta manera se conserva el historial completo de la información, cumpliendo con el requisito de baja lógica establecido en la consigna.

2.8 Capturas de pantalla del sistema funcionando

A continuación se incluyen capturas de pantalla que evidencian el funcionamiento del sistema en consola.

Menú principal del sistema

```
Output - TPI_DALESIO-Prog2-UTN-TUPaD (run)

run:

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione:
```

Figura 1 – Menú principal (Categorías, Productos, Usuarios, Pedidos)

Alta de una categoría

```
Output - TPI_DALESIO-Prog2-UTN-TUPaD (run)

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 1

--- GESTIÓN DE CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Hamburguesas
Descripción: Variedad de hamburguesas caseras con papas
Categoría creada con id: 1

--- GESTIÓN DE CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
1 - Categoría: Hamburguesas - Variedad de hamburguesas caseras con papas
```

Figura 2 – Creación de categoría desde el submenú

Alta de un producto asociado a una categoría

```

--- GESTIÓN DE PRODUCTOS ---
1. Listar (general)
2. Listar por categoría
3. Crear
4. Editar
5. Eliminar
0. Volver
Seleccione: 3
Nombre: Hamburguesa Cheddar & Bacon
Descripción: Medallón de carne premium, triple cheddar y panceta crocante
Precio: 16500.00
Stock: 15
Imagen (url o nombre de archivo):
¿Disponible? (S/N): s
Categorías disponibles:
  1 - Hamburguesas
  2 - Gaseosas
Id de la categoría: 1
Producto creado con id: 1

```

Figura 3 – Creación de producto

Creación de un pedido con detalles

```

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 4

--- GESTIÓN DE PEDIDOS ---
1. Listar
2. Crear pedido con detalles
3. Actualizar estado / forma de pago
4. Eliminar
0. Volver
Seleccione: 2
Usuarios disponibles:
  1 - Martin Dalesio
  2 - Juan Perez
Ingrese el id del usuario que realiza el pedido: 1
Forma de pago: 1) TARJETA 2) TRANSFERENCIA 3) EFECTIVO
Seleccione: 1
Productos disponibles:
  1 - Hamburguesa Cheddar & Bacon ($16500,00)
  2 - Gaseosa Regular ($5000,00)
Ingrese id del producto a agregar: 1
Cantidad: 2
Detalle agregado correctamente.
¿Agregar otro producto? (S/N): s
Productos disponibles:
  1 - Hamburguesa Cheddar & Bacon ($16500,00)
  2 - Gaseosa Regular ($5000,00)
Ingrese id del producto a agregar: 2
Cantidad: 4
Detalle agregado correctamente.
¿Agregar otro producto? (S/N): n

```



```

=====
      PEDIDO CONFIRMADO CON ÉXITO!
=====
> Pedido #1 | Cliente: Martín Dalesio | Fecha: 2026-06-22 | Estado: PENDIENTE | FormaPago: TARJETA | TOTAL: $53000,00
-----
Detalle de los productos:
-DetallePedido #1: Hamburguesa Cheddar & Bacon ($16500,00) x 2 => Subtotal: $33000,00
-DetallePedido #2: Gaseosa Regular ($5000,00) x 4 => Subtotal: $20000,00
-----
TOTAL FINAL: $53000,00
=====

```

Figura 4 – Carga de detalles de pedido y cálculo del total

Manejo de un error de validación (ejemplo: stock insuficiente)

```

--- GESTIÓN DE PEDIDOS ---
1. Listar
2. Crear pedido con detalles
3. Actualizar estado / forma de pago
4. Eliminar
0. Volver
Seleccione: 2
Usuarios disponibles:
  1 - Martín Dalesio
  2 - Juan Perez
Ingrese el id del usuario que realiza el pedido: 1
Forma de pago: 1) TARJETA  2) TRANSFERENCIA  3) EFECTIVO
Seleccione: 3
Productos disponibles:
  1 - Hamburguesa Cheddar & Bacon ($16500,00)
Ingrese id del producto a agregar: 1
Cantidad: 15
Error: Stock insuficiente para Hamburguesa Cheddar & Bacon Disponible: 5
Se cancela la creación del pedido.

```

Figura 5 – Mensaje de error ante stock insuficiente

3. Dificultades y Soluciones

Durante el desarrollo del proyecto surgieron diversas dificultades técnicas, propias del proceso de diseño de un sistema orientado a objetos con múltiples entidades relacionadas. A continuación se detallan las más relevantes y la solución adoptada en cada caso.

3.1 Generación de identificadores en clases de dominio

Inicialmente, la generación de identificadores (id) se realizaba mediante un contador estático dentro de la propia entidad (por ejemplo, un atributo static Long contador en la clase DetallePedido). Esto generaba un acoplamiento indebido entre la lógica de identidad y el modelo de dominio, además de representar un riesgo en escenarios concurrentes. La solución adoptada fue trasladar la responsabilidad de generación de identificadores a la capa de Service correspondiente, mediante un contador de instancia (no estático), pasando el id ya generado al constructor de la entidad.

3.2 Dependencia circular entre Categoría y Producto

Al implementar la validación que impide eliminar una categoría con productos activos asociados, se presentó una dependencia circular: ProductoServicio requería conocer a CategoríaServicio para validar la existencia de categorías al crear productos, y viceversa. Esto planteaba un conflicto en el orden de instanciación. La solución adoptada consistió en inyectar CategoríaServicio por constructor dentro de ProductoServicio, y resolver la referencia inversa en CategoríaServicio mediante una inyección por método setter (setProductoServicio), el cual es invocado desde la clase Main inmediatamente después de haber inicializado ambas dependencias."

3.3 Consistencia de las colecciones al crear un pedido con detalles

La consigna exige que, si ocurre un error durante la carga de detalles de un pedido (por ejemplo, falta de stock en alguno de los productos), la creación del pedido se cancele por completo sin dejar datos inconsistentes en memoria. Se resolvió diseñando un flujo de creación en tres etapas (iniciar, agregar detalles, confirmar), de modo que el objeto Pedido solo se incorpora a la colección definitiva una vez confirmado exitosamente, evitando así tener que revertir manualmente operaciones ya aplicadas sobre la colección.

3.4 Validación de unicidad de campos (nombre de categoría, mail de usuario)

Para cumplir con los criterios de aceptación que exigen unicidad (nombre de categoría, mail de usuario), se implementaron métodos de validación que recorren la colección activa correspondiente antes de cada alta o edición, excluyendo de la comparación al propio registro cuando se trata de una edición. Asimismo, se normalizó el mail a minúsculas antes de almacenarlo y compararlo, evitando que variaciones de mayúsculas/minúsculas generen falsos negativos en la validación de unicidad.

3.5 Programación Genérica para Evitar Duplicación

Inicialmente se evaluó resolver el control de colecciones vacías mediante la sobrecarga de métodos en la capa utilitaria. Pero para cumplir con el criterio de **no duplicación de código** exigido en las pautas de evaluación, decidí implementar un **método genérico unificado** en la clase Validaciones. En lugar de duplicar la estructura con sobrecarga de métodos, utilicé la sintaxis **List<? extends Base>**. Esto aprovecha la arquitectura de herencia del sistema, donde todas las entidades extienden de la clase

abstracta Base. De esta manera, el método es capaz de procesar polimórficamente colecciones de Usuarios, Categorías o Pedidos de forma segura y reutilizable, garantizando una alta cohesión.

4. Bibliografía / Webgrafía

- Oracle. (s.f.). Documentación oficial del lenguaje Java (JDK 21).
<https://docs.oracle.com/en/java/javase/21/>
- Oracle. (s.f.). The Java Tutorials – Collections. <https://docs.oracle.com/javase/tutorial/collections/>
- Oracle. (s.f.). The Java Tutorials – Exceptions.
<https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- Material y consignas de la cátedra Programación 2 – Tecnicatura Universitaria en Programación a Distancia, UTN.
- maxiprograma. (s.f.). Material del Curso C# Nivel 2 [.Net - POO – SQL] unidad 7 Arquitectura en capas.
<https://maxiprograma.com/CourseDetails?IdCurso=3>
https://drive.google.com/file/d/1It68wtz-WqIGzKiWY9vwbl_0GgqfJew9/view

5. Enlaces del Proyecto

A continuación, se detallan los enlaces solicitados para la evaluación del presente trabajo.

Repositorio GitHub:	https://github.com/dalesiomartin/TPI_DALESIO-Prog2-UTN-TUPaD.git
Video demostrativo:	https://youtu.be/gC2IGcTm7vs